

// == App Shell (js/app.js) == /*ð* vÊfz\ n8b T -Æ @aAppo8heh1077ð* @version 3.2.0ð* @date 2025-05-10 etT*01k86bhp0nyuJTyWSggYgiñi

{ "id": "tetris", "name": "0Ä•we_e1w0p '0>Ÿñ", "desc": "~IQx0Ä•weägswŷ'mIqde^L_wB:Sp0Wlé;DTetrisGame }", { "id":

{ "id": "fruitcatch", "name": "c\u00e9l\u00e9bre", "icon": "\u2639", "desc": "n\u00e9cessite de 1000000 de points pour d\u00e9bloquer", "module": "FruitCatchGame", "type": "Puzzle" }

{ "id": "whack", "name": "bSw0Y ", "con": "Ø=Ü9", "desc": "30y0-PeöbSw0Ysÿ ψ<8%bK_ř•Šø •ŠYjW0ú1ø: WhackGameø }, { "id":


```
<script src="js/games/colormatch.js"></script>
<script src="js/games/delta.js"></script>
<script src="js/games/bubble.js"></script>
<script src="js/games/colormatch.js"></script>
```



```
this.container = container;      this.statsCb = statsCb;      this.score = 0;      this.lives = 3;      this.level = 1;      this.running = t
```



```
    },    showLevelIntro(cb) {    const div = document.createElement('div');    div.className = 'level-intro';    div.innerHTML = `<div cl
```



```
if (b.y + b.r >= this.height - p.h && b.y + b.r <= this.height - p.h + 10 && b.x > p.x && b.x < p.x + p.w) { b.dy = -M
```



```
    this.showLevelIntro(() => {
        this.buildLevel();
        this.updateStats();
        this.loop = setInterval(() => this.tick()
```



```
document.removeEventListener('mousemove', this.handleMouse);  });  // === Bubble Shooter (js/games/bubble.js) === /** * Bubble Shoo
```



```
    this.loop();    });  },    startLevel() {    const cfg = this.levelConfigs[this.level - 1];    this.colors = this.allColors.slice(
```



```
color: this.colors[Math.floor(Math.random() * this.colors.length)],
row: r, col: c
});
}
}, show
```



```
    }    for (let i = 0; i < this.bubbles.length; i++) {    const b = this.bubbles[i];    const dx = cb.x - b.x;    const dy = c
```



```
if (this.bubbles.length === 0) {      if (this.level >= this.maxLevel) { this.gameOver(true); }      else { this.levelClear(); }
```



```
Puzzle Games Collection V3.2
{ row: r - 1, col: c }, { row: r - 1, col: c + 1 },
{ row: r + 1, col: c }, { row: r + 1, col: c + 1 }));
}
```



```
ctx.beginPath(); ctx.arc(x - r * 0.3, y - r * 0.3, r * 0.3, 0, Math.PI * 2); ctx.fillStyle = 'rgba(255,255,255,0.3)'; ctx.fill()
```


Puzzle Games Collection V3.2

aliens: [], bullets: [], alienBullets: [], alienDir: 1, alienSpeed: 0.5, alienDrop: 8, score: 0, lives: 3, level: 1, maxLevel: 5,


```
    this.container.style.position = 'relative';    this.container.appendChild(div);    setTimeout(() => {        if (div.parentNode) div.
```



```
});    this.canvas.addEventListener('click', () => this.shoot());    this.canvas.addEventListener('mousemove', e => {    const re
```



```
for (const b of this.bullets) {      for (const a of this.aliens) {      if (!a.alive) continue;      if (Math.abs(b.x - a.x) <
```



```
ctx.fillStyle = '#ffd93d';    for (const b of this.bullets) ctx.fillRect(b.x - 2, b.y - 6, 4, 10);    ctx.fillStyle = '#ff6b6b';
```



```
    this.loop = setInterval(() => this.tick(), this.speed);    }); },    showLevelIntro(cb) {    this.running = false;    const cfg =
```



```
const el = document.getElementById(id); if (el) el.addEventListener('touchstart', e => { e.preventDefault(); SnakeGame.s
```



```
    this.showLevelIntro(() => {          this.updateStats();          this.loop = setInterval(() => this.tick(), this.speed);          });
```


* yŮR`ziu(touch start/end)SR-niŋyused0w20pss0Hs0bŷt-GRn02p48 = { grid: [], // 4x4ep5wŋe5 0, // _SRnŋRiŋeF:^w00


```
Puzzle Games Collection V3.2
    const v = this.grid[r][c];
    cell.textContent = v || '';
    cell.className = 'cell-2048' + (v ? ` tile-${v}` : '');
  });
```


Puzzle Games Collection V3.2

```
const dy = e.clientY - Game2048.touchStart.y;    Game2048.touchStart = null;    if (Math.abs(dx) < 15 && Math.abs(dy) < 15) return;
```



```
      this.container.appendChild(div);      document.getElementById('replayBtn').addEventListener('click', () => { this.destroy(); this.ini
```



```
this.canvas = document.createElement('canvas');    this.canvas.className = 'game-canvas';    this.canvas.width = width; this.canvas
```



```
const bx = pos.x + c, by = pos.y + r;
```

```
if (bx < 0 || bx >= this.cols || by >= this.rows) return true;
```

```
if (by >= 0
```



```
},    hardDrop() {    while (!this.collides(this.piece.shape, { x: this.piecePos.x, y: this.piecePos.y + 1 })) this.piecePos.y++;    t
```



```
countMines(r, c) {  
    let count = 0;  
    for (let dr = -1; dr <= 1; dr++)  
        for (let dc = -1; dc <= 1; dc++) {  
            const nr = r + dr;  
            const nc = c + dc;  
            if (nr < 0 || nr > 9 || nc < 0 || nc > 9) continue;  
            if (board[nr][nc] === 'M') count++;  
        }  
    return count;  
}
```



```
    } else {          // Flood fill          for (let dr = -1; dr <= 1; dr++)          for (let dc = -1; dc <= 1; dc++) {          const nr = r +
```



```
});    const div = document.createElement('div');    div.className = 'game-over-overlay';    div.innerHTML = `    <h3>${won ? '0'}
```



```
initGrid() {
  this.grid = Array.from({ length: this.size }, () =>
    Array.from({ length: this.size }, () =>
      this.gems[Mat
```


const r = parseInt(cell.dataset.r);

const c = parseInt(cell.dataset.c);

this._touchStart = { r, c, x: e.touches[0].x, y: e.touches[0].y };


```
    this.swap(r1, c1, r2, c2);  
    await this.animateSwap(r1, c1, r2, c2);  
    const matches = this.findAllMatches();  
    if (matches.length > 0) {
```



```
    if (this.moves <= 0) this.gameOver(); }, swap(r1, c1, r2, c2) {    const tmp = this.grid[r1][c1];    this.grid[r1][c1] = this.grid[r2][c2];    this.grid[r2][c2] = tmp;    this.moves--;
```



```
let end = r + 2;Ø
```

```
while (end + 1 < this.size && this.grid[end + 1][c] === this.grid[r][c]) end++;Ø
```

```
for (let i
```

